מבוא למדעי המחשב - 67102 - סמסטר קיץ 2022
מועד א' - 22.9.22

<u>חלק א' - הצעות לפתרון:</u>

.1

```
O(n^3)
```

.2

```
O((n^2)*logn)
```

.3

```
'AWBWWDF' - (3,4), (3,'4')
'AWMBWWDF' - ('3',4)
'WCF' - ('3','4')
'WMCD' - ('3',[4])
```

.4

```
7
```

.5

```
(1) f(v) < f(w)
(2) v in s
(3) w in s
```

.6

```
(6) sorted(s, key=lambda x:(f(x),x))
```

.7

```
(1) range(len(x))
(2) len(x)==1
(3) x[1:], m+x[0]

OR:

(1) [0]
(2) len(x)==1
(3) [sum(x)], 0
```

.8

```
(1) BinT
(2) mid
(3) buildBST
(4) :mid
(5) mid+1:
```

.9

```
(1) a or b
(2) not b or a and a[0] < b[0]
(3) yield b.pop(0)
```

.10

```
(1) t[len(p):]
(2) t[len(p):]
(3) @a
(4) @b
```

<u>חלק ב' - הצעות לפתרון:</u>

.11

```
def digadder(a, b, c):
    return (a + b + c) % 10, (a + b + c) // 10

def sum_lists(list_1, list_2):
    c=0
    res=[]
    for i in range(1, max(len(list_1), len(list_2)) + 1):
        a = 0 if i > len(list_1) else list_1[-i]
        b = 0 if i > len(list_2) else list_2[-i]
        x, c = digadder(a, b, c)
        res.insert(0, x)
    if c:
        res.insert(0, c)
    return res
```

.12

```
def cut(node):
    p,n=node.prev,node.next.next
    if p:
        p.next=n
    if n:
        n.prev=p
    return p if p else n

def balanced(head):
    while head:
        if head.data==')' or not head.next:
            return False
        if head.next.data=='(':
            head=head.next
        else: #head.next.data==')'
            head=cut(head)
    return True
```

.13

```
def neighbors(mat, place):
    y, x = place
    return {(y + dy, x + dx)
            for dy in [-1, 0, 1] for dx in [-1, 0, 1]
            if dx or dy
            if 0 <= y + dy < len(mat)
            if 0 <= x + dx < len (mat[0])}


# Option 1:

def find_stain_size(mat, place):
    visited = set()
    Queue = {tuple(place)}
    while queue:
        y, x = queue.pop()
        if (y, x) in visited or mat[y][x] == 0:
            continue
        visited.add((y, x))
        queue |= neighbors(mat,(y, x))
    return len(visited)

# Option 2:

def find_stain_size(mat, place):
    Y, x = place
    if mat[y][x] != 1:
        return 0
    mat[y][x] = 2
    return 1 + sum(find_stain_size(p) for p in neighbors(mat, place)
```

.14

```
def pair_sums(d):
    if not d:
        return True
    target = max(d) + min(d)
    for key, count in d.items():
        comp = target - key
        if comp not in d or d[comp] != count or comp == key and count % 2:
            return False
    return True
```

.15

```
class BinT:
    def __init__(self, data, left=None, right=None):
        self.data = data
        self.left = left
        self.right = right
```

```
def trim(root, low, high):
    if not root:
        return None
    if high is not None and root.data >= high:
        return trim(root.left, low, high)
    if low is not None and root.data <= low:
        return trim(root.right, low, high)
    if high is not None:
        root.right = trim(root.right, None, high)
    if low is not None:
        root.left=trim(root.left, low, None)
    return root
```

.16

```
from functools import total_ordering


@total_ordering
class Time:
    def __init__(self, hours: int, minutes: int) -> object:
        if not (0 <= hours < 24) or not (0 <= minutes < 60):
            raise ValueError("Invalid Input Time")
        self.hours, self.minutes = hours, minutes

    def __eq__(self, other):
        return (isinstance(other, type(self)) and
                (self.hours, self.minutes) == (other.hours, other.minutes))

    def __le__(self, other):  # allow objects to be compared
        return (self.hours, self.minutes) <= (other.hours, other.minutes)

    def __repr__(self):  # represent object as string
        return f'{self.hours}:{self.minutes}'


class Schedule:
    def __init__(self):
        self.time_table = []

    def add_time(self, t):
        if t in self.time_table:
            raise Exception(f"The schedule already has a train leaving at time
                            {t.hours}:{t.minutes}")
        self.time_table.append(t)
        self.time_table = sorted(self.time_table)

    def find_train(self, t):
        if not self.time_table:
            raise Exception("No train found")
        if t >= self.time_table[-1]:
            return self.time_table[0]
```

```
first = 0
last = len(self.time_table) - 1
found = False
pos = 0

while first <= last and not found:
    midpoint = (first + last) // 2
    if self.time_table[midpoint] > t >= self.time_table[midpoint - 1]:
        pos = midpoint
    else:
        if t < self.time_table[midpoint]:
            last = midpoint - 1
        else:
            first = midpoint + 1
if found:
    return self.time_table[pos]
```